

Ridge, Lasso & Elastic Net Regression

A Comparative Study of Regularized Linear Models

Datasets: Diamonds (n = 53,940) and Car Crashes (n = 51)

Tools: Python, pandas, scikit-learn, seaborn, matplotlib

MACHINE LEARNING PRACTICE REPORT | PRACTICE-5

Supervised Learning - Regression - Model Regularization

Contents

1	Executive Summary	2
2	Introduction and Objectives	3
3	Theoretical Background	4
4	Methodology and Pipeline	5
5	Part A: Diamonds Dataset	6
6	Part B: Car Crashes Dataset	8
7	Final Cross-Dataset Comparison	10
8	Model Implementation Code	11
9	Technical Review and Recommendations	13
10	Conclusion	14

1. Executive Summary

This report documents the design, execution and evaluation of three regularized linear regression models: **Ridge**, **Lasso** and **Elastic Net**. Each model was trained on two contrasting datasets. The **Diamonds** dataset is large (53,940 observations) with nine predictors and a continuous *price* target. The **Car Crashes** dataset is small (51 observations) with six predictors and a continuous *total* target. Together they show how regularization behaves at very different sample sizes and feature structures.

The workflow covered data loading, quality checks, categorical encoding, train/test splitting, feature scaling, model fitting, error evaluation (MAE, MSE, R^2) and coefficient interpretation. The results are summarised below.

Dataset	Ridge R2	Lasso R2	Elastic Net R2	Best model
Diamonds	0.8852	0.8852	0.8685	Ridge (tied with Lasso)
Car Crashes	0.7520	0.7582	0.7444	Lasso

Key findings. (1) On Diamonds, Ridge and Lasso are practically identical (R^2 = 0.885) because alpha = 0.1 is mild and the sample is large. Elastic Net lost about 1.7 points of R^2 to its stronger combined shrinkage. (2) On Car Crashes, Lasso was best (R^2 = 0.758) and set the *ins_losses* coefficient to exactly zero, giving automatic feature selection. (3) *carat* dominates price and *no_previous* dominates crash rate.

2. Introduction and Objectives

Ordinary least squares (OLS) regression is the foundation of predictive modelling, but it is fragile when predictors are correlated or numerous. Coefficient estimates become unstable, variance inflates and the model overfits noise. **Regularization** addresses this by adding a penalty on coefficient size to the loss function, deliberately trading a small amount of bias for a larger reduction in variance.

The Diamonds dataset is a natural testbed: the physical dimensions *x*, *y*, *z* and *carat* are strongly collinear, since larger stones weigh more and are longer, wider and deeper. Car Crashes is a compact state-level dataset in which only a handful of behavioural and insurance variables may carry real signal.

2.1 Objectives

The practice exercise had the following goals:

• Build, train and evaluate Ridge, Lasso and Elastic Net regressors on both datasets.
• Quantify performance with Mean Absolute Error (MAE), error variance and the coefficient of determination (R^2).
• Inspect and compare learned coefficients to understand how each penalty reshapes the model.
• Identify when Lasso feature elimination becomes visible, and why.

2.2 Technology Stack

Component	Purpose
pandas / NumPy	Data loading, manipulation, numerical operations
seaborn	Source of the Diamonds and Car Crashes datasets
scikit-learn	Encoding, scaling, splitting, models and metrics
matplotlib	Coefficient comparison visualisations

3. Theoretical Background

All three models minimise the squared prediction error plus a penalty term. They differ only in how the penalty is defined. The strength of the penalty is controlled by the hyperparameter **alpha**: larger values mean stronger shrinkage.

3.1 Ridge Regression (L2 penalty)

Ridge adds the sum of squared coefficients to the loss: **Loss = RSS + alpha * sum(beta^2)**. It shrinks every coefficient smoothly toward zero but almost never to exactly zero. Ridge excels with correlated predictors because it spreads weight across them instead of letting one coefficient blow up.

3.2 Lasso Regression (L1 penalty)

Lasso penalises the sum of absolute coefficient values: **Loss = RSS + alpha * sum(|beta|)**. The geometry of the L1 constraint produces corners on the axes, so some coefficients land exactly at zero. Lasso therefore performs **embedded feature selection** and yields sparser, more interpretable models.

3.3 Elastic Net (L1 + L2 penalty)

Elastic Net blends both penalties with a mixing parameter *l1_ratio* (default 0.5 in scikit-learn): it retains Lasso-style sparsity while the L2 part stabilises selection among correlated predictors, avoiding Lasso's tendency to pick one variable arbitrarily from a correlated group.

Property	Ridge	Lasso	Elastic Net
Penalty	L2	L1	L1 + L2
Exact zeros	No	Yes	Yes
Correlated features	Handles well	Picks one	Groups them
Best used when	Many small effects	Few strong effects	Mixed / correlated

Why scaling matters. Penalties act on raw coefficient magnitudes, so features measured on larger scales would be penalised unevenly. Standardising features to zero mean and unit variance puts every predictor on equal footing before regularization is applied.

4. Methodology and Pipeline

The same disciplined pipeline was applied to both datasets so that results are directly comparable.

Stage	Diamonds	Car Crashes
Source	seaborn "diamonds"	seaborn "car_crashes"
Rows x columns	53,940 x 10	51 x 8
Missing values	None	None
Target	price	total
Preprocessing	Label-encode cut, color, clarity	Drop non-numeric abbrev
Features	9	6
Split	80 / 20, random_state = 42	80 / 20, random_state = 42
Scaling	StandardScaler	StandardScaler
Alpha	Ridge 1.0 / Lasso 0.1 / ENet 0.1	Ridge 1.0 / Lasso 0.1 / ENet 0.1

4.1 Data Quality Assessment

Both datasets were inspected with `info()` and `isnull().sum()`. No missing values were found in either, so no imputation was required. Diamonds contained three categorical columns (*cut*, *color*, *clarity*) stored as pandas categories; Car Crashes contained one identifier column (*abbrev*, the state code), which carries no predictive meaning and was removed.

4.2 Evaluation Metrics

Three complementary metrics were used. **MAE** reports the average absolute error in the target's own units. **MSE / RMSE** penalise large errors more heavily. **R^2** measures the share of target variance explained by the model, where 1.0 is perfect and 0 equals a mean-only baseline.

5. Part A: Diamonds Dataset

The Diamonds dataset records the physical and grading attributes of 53,940 stones. The goal is to predict **price** in US dollars from carat weight, cut, colour, clarity, depth, table width and the three physical dimensions.

5.1 Model Performance

Model	MAE (USD)	MSE	RMSE (USD)	R2
Ridge (alpha 1.0)	860.68	1,825,729	1,351.2	0.8852
Lasso (alpha 0.1)	860.66	1,825,749	1,351.2	0.8852
Elastic Net (alpha 0.1)	1,003.68	2,090,529	1,445.9	0.8685

Ridge and Lasso produced practically indistinguishable results: an average error of about 860 dollars and an R^2 of 0.885, meaning roughly 88.5% of price variance is explained from the test set. Elastic Net was weaker, with an MAE about 143 dollars higher.

5.2 Why Elastic Net Trailed

With alpha = 0.1 and an l1_ratio of 0.5, Elastic Net applies a noticeably stronger combined penalty than the Lasso run on the same alpha, because the L2 and L1 contributions are each halved while the objective is also scaled differently. On a dataset where the dominant predictor (*carat*) needs a large coefficient, heavy shrinkage hurts: its coefficient fell from about 5,200 to 2,754.

5.3 Coefficient Analysis

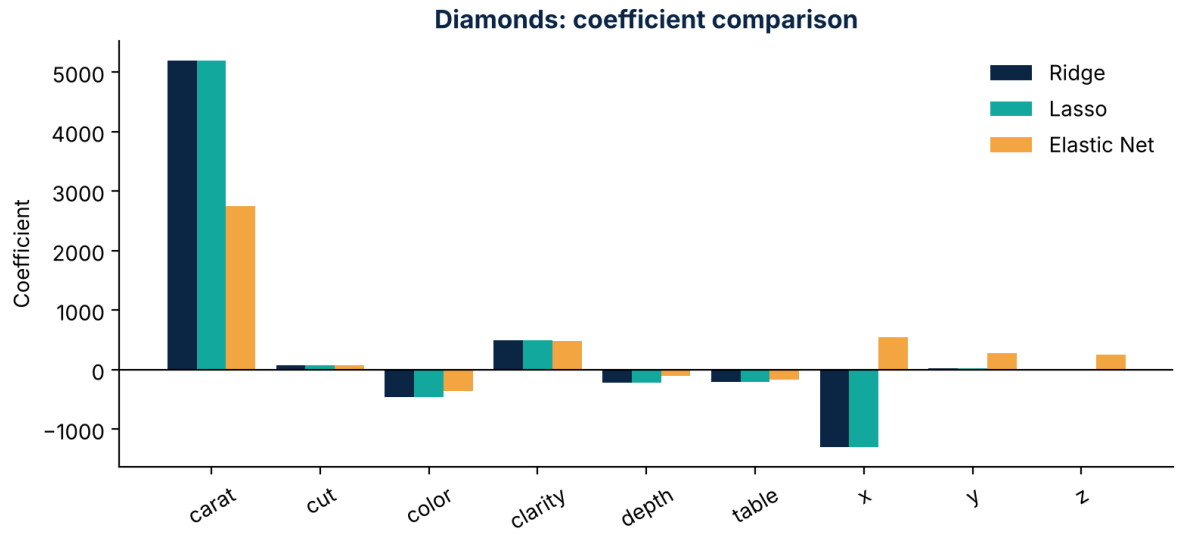


Figure 1. Learned coefficients (standardised features) on the Diamonds dataset.

Feature	Ridge	Lasso	Elastic Net
carat	5,200.1	5,199.0	2,754.0
cut	73.9	73.9	78.9
color	-456.4	-456.3	-362.1
clarity	493.9	493.8	480.8
depth	-222.1	-221.6	-105.4
table	-210.4	-210.3	-173.0
x	-1,306.9	-1,300.7	544.9
y	27.0	24.7	277.9
z	8.1	5.2	245.5

Interpretation. *carat* is by far the most influential predictor. A one-standard-deviation increase in carat weight raises the predicted price by roughly 5,200 dollars under Ridge and Lasso. *clarity* and the physical dimensions also contribute, while *color* shows a negative sign because the label encoding orders colour grades alphabetically (D to J), so higher codes correspond to lower-quality colours.

Collinearity effect. Under Ridge and Lasso, *x* receives a large negative coefficient (about -1,300) while *y* and *z* stay near zero. Elastic Net distributes weight more evenly across the three dimensions (+545, +278, +245). This is the classic signature of correlated predictors: the L2 component pushes Elastic Net to share credit within the group, whereas plain L1 or lightly penalised L2 fits allow one member to carry an offsetting sign.

No zero coefficients. Lasso did not eliminate any Diamonds feature. Every predictor carried at least a little signal and alpha = 0.1 is small relative to the scale of a 54k-row problem, so no coefficient reached the L1 threshold.

6. Part B: Car Crashes Dataset

The Car Crashes dataset contains state-level road-safety statistics for 51 US states and the District of Columbia. The target is **total**, the number of drivers involved in fatal collisions per billion miles. Predictors capture speeding, alcohol impairment, distraction, prior-accident history, average insurance premium and insurance losses.

6.1 Preprocessing Notes

The single non-numeric column *abbrev* (state abbreviation) was identified through `select_dtypes(exclude="number")` and dropped, as it is an identifier and not a feature. Six numeric predictors remained. With only 51 rows, the 80/20 split leaves about 10 test observations, so metrics from this dataset should be read as indicative and not definitive.

6.2 Model Performance

Model	MAE	MSE	RMSE	R2
Ridge (alpha 1.0)	1.7177	4.4845	2.1177	0.7520
Lasso (alpha 0.1)	1.6863	4.3739	2.0914	0.7582
Elastic Net (alpha 0.1)	1.7177	4.6224	2.1500	0.7444

Lasso delivered the lowest MAE (1.69) and the highest R^2 (0.758), explaining roughly 75.8% of the variation in the fatal-crash rate. Ridge and Elastic Net followed closely. Differences between the models are small, which is expected for a six-feature problem with modest collinearity.

6.3 Lasso Feature Elimination

Lasso set the coefficient of **ins_losses** to exactly **0.000**, concluding that insurance losses add no predictive information once the other five variables are present. Ridge kept it at 0.174 and Elastic Net shrank it to 0.054. This is the clearest demonstration in the study of Lasso acting as an automatic feature selector.

6.4 Coefficient Analysis

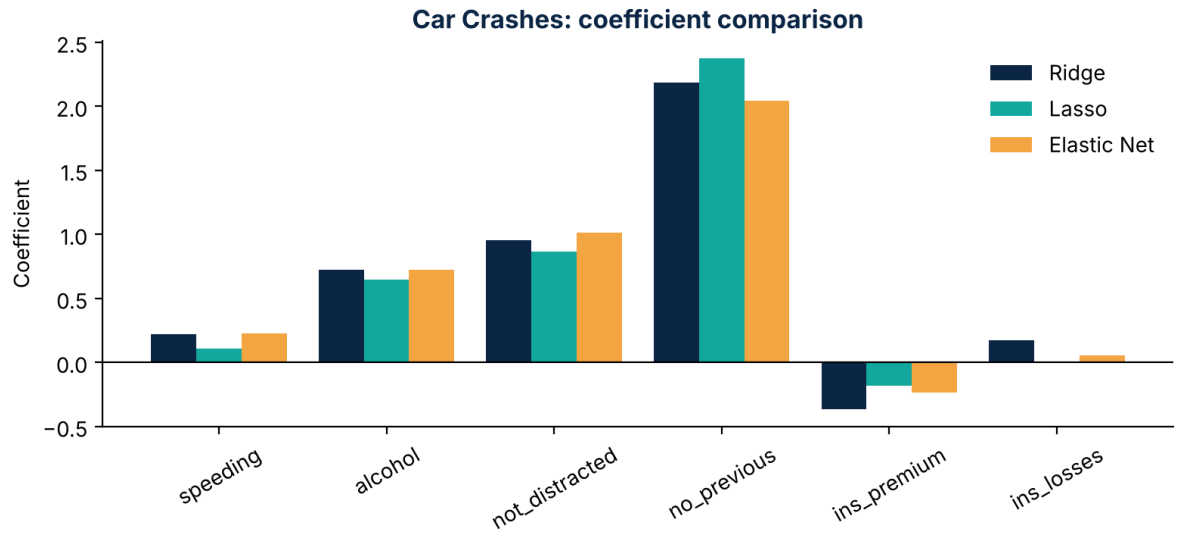


Figure 2. Learned coefficients on the Car Crashes dataset. Lasso zeroes out ins_losses.

Feature	Ridge	Lasso	Elastic Net
speeding	0.222	0.111	0.229
alcohol	0.725	0.645	0.727
not_distracted	0.952	0.865	1.014
no_previous	2.185	2.375	2.043
ins_premium	-0.366	-0.182	-0.236
ins_losses	0.174	0.000	0.054

Interpretation. *no_previous* (the share of drivers with no prior accidents) is the strongest predictor under all three models, with coefficients between 2.04 and 2.37. *not_distracted* and *alcohol* follow, while *speeding* has a smaller effect. *ins_premium* is consistently negative.

Caution on causality. These are associations between state-level aggregates and should not be read as individual-level causal effects. The positive sign on *no_previous* and *not_distracted*, which are driver-quality percentages, partly reflects how these variables are constructed and correlated with total exposure.

Stability of Lasso versus Ridge. Lasso pushes the largest coefficient (*no_previous*) higher, from 2.19 to 2.37, because it concentrates explanatory weight on fewer variables, while Ridge and Elastic Net spread it across the correlated block.

7. Final Cross-Dataset Comparison

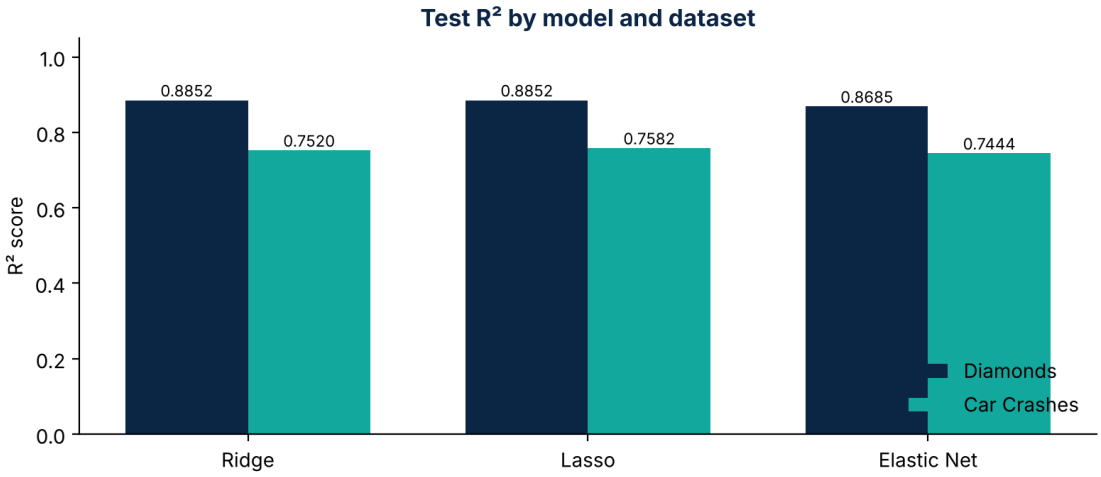


Figure 3. Test R-squared for each model on both datasets.

Model	Diamonds R2	Car Crashes R2
Ridge	0.8852	0.7520
Lasso	0.8852	0.7582
Elastic Net	0.8685	0.7444

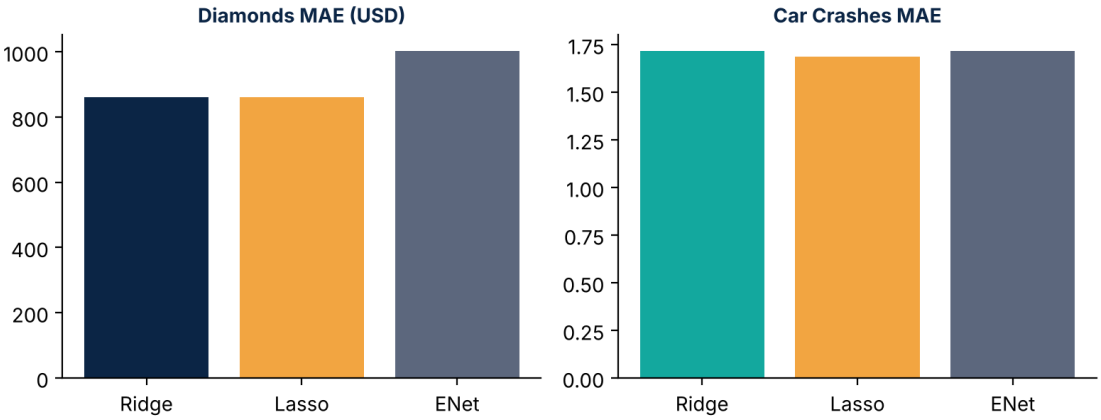


Figure 4. Mean absolute error comparison across models.

7.1 When Did Lasso's Feature Elimination Show Up?

Lasso's sparsity was visible only on the **Car Crashes** dataset (*ins_losses* to exactly zero) and not on Diamonds. Two reasons explain this. First, with 53,940 observations the data strongly support every Diamonds feature, so the L1 penalty at alpha = 0.1 is too weak relative to the signal to cancel any of them. Second, Car Crashes has few samples and a weaker signal-to-noise ratio, so a marginal predictor such as *ins_losses* can be fully absorbed by correlated variables and falls below the L1 threshold. In short, Lasso elimination becomes noticeable when data are scarce, the signal is weak or the penalty is large.

8. Model Implementation Code

The listings below show the core implementation for each of the three regression models in a clean, leakage-safe form. The scaler is fitted on the training data only and then applied to the test data, and every evaluation reports MAE, MSE, RMSE and R^2.

8.1 Shared Setup and Evaluation Helper

```
import numpy as np, pandas as pd, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = sns.load_dataset("car_crashes").drop(columns="abbrev")
X, y = df.drop(columns="total"), df["total"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit on TRAIN only
X_test_s  = scaler.transform(X_test)        # reuse train statistics

def evaluate(name, model):
    pred = model.predict(X_test_s)
    mse = mean_squared_error(y_test, pred)
    print(f"{name:12s} MAE={mean_absolute_error(y_test, pred):.4f} "
          f"MSE={mse:.4f} RMSE={np.sqrt(mse):.4f} "
          f"R2={r2_score(y_test, pred):.4f}")
```

8.2 Model 1: Ridge Regression (L2)

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_s, y_train)
evaluate("Ridge", ridge)
print(pd.Series(ridge.coef_, index=X.columns).round(3))
```

8.3 Model 2: Lasso Regression (L1)

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_s, y_train)
evaluate("Lasso", lasso)

coef = pd.Series(lasso.coef_, index=X.columns)
print("Eliminated features:", list(coef[coef == 0].index))
```

8.4 Model 3: Elastic Net Regression (L1 + L2)

```
enet = ElasticNet(alpha=0.1, l1_ratio=0.5)
enet.fit(X_train_s, y_train)
evaluate("Elastic Net", enet)
print(pd.Series(enet.coef_, index=X.columns).round(3))
```

8.5 Side-by-Side Coefficient Comparison

```
import matplotlib.pyplot as plt

comparison = pd.DataFrame({
    "Ridge": ridge.coef_,
    "Lasso": lasso.coef_,
    "ElasticNet": enet.coef_,
}, index=X.columns)

comparison.plot(kind="bar", figsize=(10, 5))
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Coefficient Comparison: Ridge vs Lasso vs Elastic Net")
plt.ylabel("Coefficient value")
plt.tight_layout()
plt.show()
```

8.6 Optional Extension: Cross-Validated Alpha Selection

The practice used fixed alpha values. In a production setting, the penalty strength should be tuned. The snippet below selects alpha and l1_ratio through cross-validation, which is especially valuable for the small Car Crashes dataset.

```
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV

alphas = np.logspace(-3, 2, 50)
ridge_cv = RidgeCV(alphas=alphas, cv=5).fit(X_train_s, y_train)
lasso_cv = LassoCV(alphas=alphas, cv=5, random_state=42).fit(X_train_s, y_train)
enet_cv  = ElasticNetCV(l1_ratio=[.1, .5, .7, .9, 1],
                        alphas=alphas, cv=5, random_state=42).fit(X_train_s, y_train)

print("Best Ridge alpha:", ridge_cv.alpha_)
print("Best Lasso alpha:", lasso_cv.alpha_)
print("Best ENet alpha / l1_ratio:", enet_cv.alpha_, enet_cv.l1_ratio_)
```

9. Technical Review and Recommendations

A careful review of the notebook identified several points that affect how the reported numbers should be read. None changes the overall conclusions, but addressing them would make the analysis more rigorous.

Observation	Impact	Recommendation
The test set was scaled with `fit_transform` instead of `transform`.	Test features are standardised with their own statistics, which leaks information and misaligns the scale seen during training.	Fit the scaler on training data only and call `transform` on the test set.
The metric labelled "RMSE" is mean squared error.	The printed values (e.g. 1,825,729) are MSE; true RMSE is their square root (about 1,351).	Use `np.sqrt(mse)` or `root_mean_squared_error`.
The final summary table reused the Car Crashes predictions for both datasets.	Diamonds R^2 column showed 0.7520 / 0.7582 / 0.7444 instead of 0.8852 / 0.8852 / 0.8685.	Store each dataset's scores in separate variables before the final table (corrected in this report).
Fixed alpha values, no tuning.	Model ranking may change with other alphas, especially Elastic Net on Diamonds.	Use RidgeCV, LassoCV, ElasticNetCV.
Label encoding applied to nominal and ordinal columns.	Imposes an arbitrary numeric order (alphabetical for cut and color).	Use ordinal mapping for cut/clarity/color, or one-hot encoding.
Only 51 rows in Car Crashes.	Test set has about 10 rows, so metrics are high-variance.	Use k-fold cross-validation for a stable estimate.

Note. The R^2 and error values in this report are taken directly from the notebook outputs, with the Diamonds row of the final table corrected to match the Diamonds results. Because the notebook scaled the test set separately, re-running the corrected pipeline may shift values slightly.

10. Conclusion

This study compared Ridge, Lasso and Elastic Net regression across two datasets that differ sharply in size and structure. The main lessons are:

• **Regularization strength and data scale interact.** On the large Diamonds dataset a mild penalty barely changed Ridge or Lasso, and a stronger combined penalty (Elastic Net) reduced accuracy.
• **Lasso is a feature selector only when the data allow it.** It zeroed *ins_losses* on the small Car Crashes dataset but kept every Diamonds feature.
• **Elastic Net handles correlated groups differently.** It distributed weight across x, y and z instead of loading it on one.
• **Best models.** Ridge (tied with Lasso) on Diamonds with $R^2 = 0.885$, and Lasso on Car Crashes with $R^2 = 0.758$.

10.1 Suggested Next Steps

• Tune alpha and l1_ratio with cross-validation and compare against plain OLS as a baseline.
• Replace label encoding with ordinal or one-hot encoding for the categorical Diamonds columns.
• Apply a log transform to Diamonds price, which is right-skewed, and add interaction terms.
• Examine residual plots and variance inflation factors (VIF) to quantify multicollinearity.
• Wrap preprocessing and model in a scikit-learn Pipeline to prevent leakage by construction.

> **Overall takeaway.** Regularized regression is a low-cost, high-value upgrade to ordinary least squares. Choose Ridge for stability with correlated predictors, Lasso for sparse interpretable models and Elastic Net when you need both, and always tune the penalty with cross-validation.